HVD 1.5 — Recursive Cognitive Runtime Foundation + TMS Persistence Hardening

Revised: 2026-08-26
Status: Active Sprint
Supersedes: The architectural sequencing and Nexus/distribution sections of `docs/notes/current_sprint-8-25-2026.txt`
Prerequisite status: HTNLA closed; engineering-debt/test-artifact cleanup complete.

---

0. Why This Sprint Has Been Rewritten

The previous sprint correctly identified several immediate engineering needs:

- TMS boundary hardening;
- staged HDF5 persistence;
- scalable integrity verification;
- explicit maintenance/readiness state;
- deterministic distribution primitives;
- separation of ANN from authoritative state.

Those requirements remain valid.

What has changed is our understanding of **what must exist before HVD can be distributed correctly**.

The previous sprint attempted to move almost directly from single-node persistence hardening into:

```
node identity
membership
placement
shards
replicas
rebalance
snapshot shipping
anti-entropy
```

That sequencing is now incomplete.

Before physical distribution can be made canonical, HVD must first define the logical object being distributed.

The updated architecture establishes that HVD is a **recursive cognitive database** operating within Nexus cognitive fields. Ephemeral HVDs are not parent/child databases. They are independent logical database state-spaces that may coexist in a dynamically evolving hyperdimensional field, temporarily associate with other HVDs, participate in multiple cognitive coalitions simultaneously, diverge, disappear, recur, or become longer-lived.

Therefore, before implementing distributed placement, HVD needs explicit answers to:

```
What is an ephemeral HVD?

How is it identified locally?

How is it identified globally while still ephemeral?

How is it referenced by another HVD?

What owns the active cognitive namespace?

What owns physical realization?

How is an HVD created without creating a heavyweight database engine?

How does one HVD share state with another without implying inheritance?

How are 100,000 logical HVDs created and reclaimed efficiently?

How does a logically complete HVD recursively execute Nexus cognition
without physically duplicating Nexus?

How does TMS remember changing relationships without converting
history into identity?

What survives when an ephemeral cognitive field disappears?
```

Until those contracts exist, distributed placement would be premature.

The revised sprint therefore has two goals:

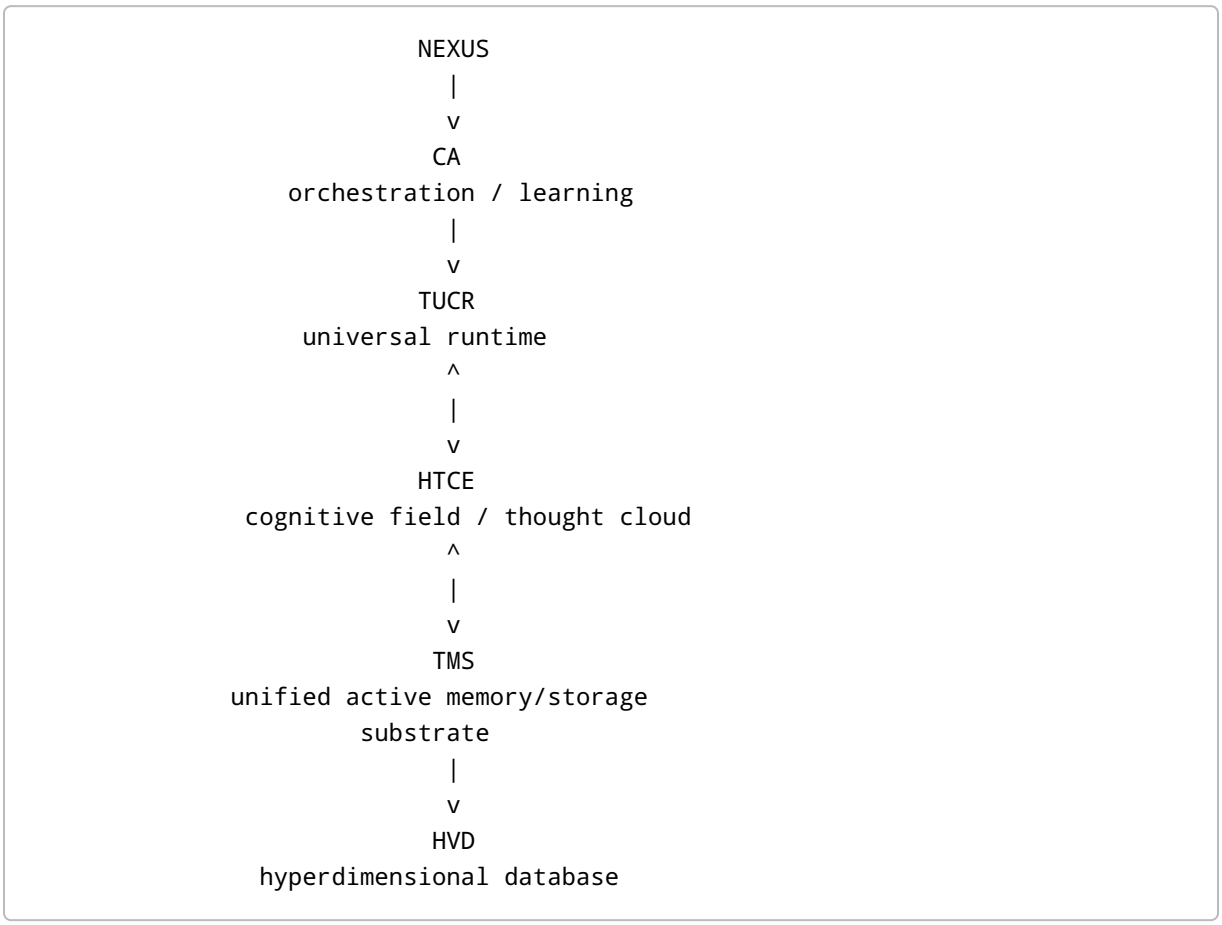
1. **Finish the single-node TMS/persistence hardening already underway.**
2. **Establish the local recursive cognitive-runtime foundation that distribution will later realize physically.**

Distribution is not abandoned.

It becomes the next sprint, where it can be built against stable logical semantics rather than speculative ones.

1. Canonical Architecture for This Sprint

The approximate framework-wide cognitive relationship is:



This diagram represents responsibility and broad control direction.

It is **not** a one-way pipeline.

The complete system contains feedback loops.

1.1 CA

CA remains responsible for high-level cognitive orchestration, prediction, learning, and system-level cognitive behavior.

CA is not:

- the storage engine;
- the cognitive object resolver;
- the filesystem;
- the distributed placement system.

1.2 TUCR

TUCR provides the universal runtime/substrate context.

It participates in:

- substrate abstraction;
- runtime compatibility;
- cognitive execution context;
- transformation between physical substrates.

TUCR does not own TMS storage extents or HTCE thought-cloud membership.

1.3 HTCE

HTCE owns the **active cognitive field**.

That includes:

```
ephemeral HVD generation
active thought-space population
cognitive addressability
candidate/coalition membership
resonance
activation/decay
field expansion
field collapse
candidate release
```

HTCE answers:

> What cognitive state-spaces currently exist in this field, and how are they participating in cognition?

HTCE does not own:

```
WAL layout
Memtable layout
SSTable extents
filesystem paths
HDF5 chunks
replica bytes
physical disk placement
```

1.4 TMS

TMS is the **Transdimensional Memory Substrate**.

It is the mandatory live substrate.

TMS should be understood as a unified cognitive storage continuum rather than merely as a conventional database backend.

At different times and for different state, it may behave:

```
RAM-like
cache-like
COW-filesystem-like
log-structured
device-backed
distributed
```

without becoming separate competing storage engines.

TMS owns:

```
active state realization
trajectory
history
working memory
structural sharing
COW state
WAL
Memtables
SSTables
compaction / reconsolidation
physical state movement
```

```
storage locality
future distributed realization
```

TMS does **not** own:

```
HTCE collapse
epistemic truth
constitutional admissibility
cognitive authority
```

A useful shorthand is:

> **HTCE owns cognitive addressability. TMS owns substrate addressability.**

1.5 HVD

HVD remains a complete database.

HVD provides:

```
database semantics
collections
records
stable logical identity
mutation
exact retrieval
associative retrieval
HTLA / HTNLA representations
indexing
cross-HVD references
durability contracts
distribution contracts
```

An HVD is not merely an HTCE thought vector.

An ephemeral HVD remains logically a real database even if its initial physical realization costs only a descriptor plus shared TMS state.

Therefore:

> **DATABASE SEMANTICS != DATABASE WEIGHT**

1.6 HIGC

HIGC owns admissible geometry / constitutional constraints.

In the soup-kitchen analogy:

```
don't fight
don't steal
don't threaten people
```

is closer to HIGC than TMS.

This sprint does **not** attempt to implement the complete HIGC architecture.

It must, however, leave correct boundaries/hooks for it.

1.7 THDE

THDE owns bounded state evolution and hysteretic intervention.

Example:

```
Bob and Steve repeatedly fight for three months
        |
        v
historical pressure accumulates
        |
        v
future service schedules diverge
```

That intervention is not equivalent to declaring:

```
Bob.enemy_of(Steve) = TRUE forever
```

TMS remembers the trajectory.

THDE can later use trajectory to influence bounded future behavior.

Again, full THDE implementation is outside this sprint, but this sprint must not create structures that make THDE impossible.

1.8 QMT

QMT owns continuity across transformation.

Future examples include:

```
shared -> isolated
local -> distributed
TMS -> HDF5 -> TMS
hardware A -> hardware B
```

Physical movement must not silently redefine cognitive identity.

This sprint must preserve enough identity and state boundaries for future QMT verification.

1.9 HDF5

HDF5 remains HVD's durable long-term/permanent persistence layer.

Do not reduce it conceptually to:

```
snapshot storage only
```

Snapshotting, staging, restoration, System Rebirth, and rehydration are mechanisms connecting TMS and HDF5.

HDF5 is **not** the live hot query engine.

1.10 ANN

ANN remains derived acceleration state.

```
TMS authoritative state
        |
        v
derived ANN representation
```

Never:

```
nearest neighbor
        |
        v
automatic truth
```

Similarity is not:

```
truth
identity
continuity
authority
admissibility
```

---

2. Non-Negotiable Recursive-HVD Semantics

Before touching code, the following architecture is binding for this sprint.

2.1 No parent/child database ontology

Do not introduce or depend upon canonical semantics such as:

```
parent_database_id
child_database_id
inheritance_mode
database ancestry
database family tree
```

Recursive cognition is not a genealogical database tree.

2.2 Recursive cognition is field-based

At cognitive time `t`, think approximately:

```
F(t) = { HVD_i(t) }
```

where the active population changes continuously.

Conceptually:

```
                ACTIVE COGNITIVE FIELD

       HVD-A                 HVD-B
          \                  /
           \   coalition α  /
            \              /
             --------------


                 HVD-C -------- HVD-D
                    coalition β

       HVD-E

       currently participates in neither coalition
```

Later:

```
A disappears
B associates with E
C becomes independent
D disappears
F, G, H appear
```

No genealogy is required.

2.3 Dynamic relationships are not automatically permanent edges

The same HVD may simultaneously participate in several contexts.

Relationships can change:

```
friend
  |
enemy
  |
friend
  |
enemy
```

```
  |
friend
```

while unrelated slower-changing state remains stable.

The architecture therefore needs:

```
current state
+
trajectory
+
context
```

rather than permanent relational facts for every cognitive interaction.

Graph representations may exist as derived/control-plane views where useful.

They are not the canonical cognitive ontology.

2.4 Structural sharing is not inheritance

A candidate HVD may share immutable TMS extents with another state.

Conceptually:

```
D_i = B + Delta_i
```

where:

```
B       = shared immutable state
Delta_i = candidate-specific divergence
```

This means:

> These bytes/state do not need to be duplicated.

It does **not** mean:

> D_i is semantically the child of B.

2.5 No self-declared permanence

An ephemeral HVD must never acquire persistence merely because its internal state says:

```
I am important.
I am true.
I should survive.
```

Stored propositions do not grant architectural authority.

Materialization/retention is externally controlled by the cognitive framework and storage lifecycle.

---

3. Sprint Objective

By the end of this sprint, HVD should have:

```
DONE
    clean lint/test baseline

PERSISTENCE HARDENING
    TMS collection-name safety
    short-barrier HDF5 staging
    scalable archive integrity
    explicit runtime readiness/maintenance state

RECURSIVE COGNITIVE FOUNDATION
    canonical recursive-runtime contracts
    local ephemeral identity
    global ephemeral identity
    CognitiveArena lifecycle
    HTCE active cognitive-field registry
    TMS structural-sharing/COW contract
    cross-HVD reference resolution
    lightweight recursive NexusScope contract
    trajectory / authority separation tests

NOT YET
    full cluster placement
    full distributed TMS
    replicated mutation
    Raft authority
```

```
    cluster-wide ANN fanout
    complete dream runtime
```

This sprint establishes the **logical architecture** that the following distributed sprint will physically realize.

---

4. Sprint Item #1 — Engineering Cleanup

STATUS: COMPLETE

Do not reopen this item unless a regression appears.

The completed work established:

```
ruff check hvd tests
    -> clean

pytest -q
    -> successful

git diff --check
    -> clean
```

Tests no longer recreate or mutate tracked:

```
benchmark artifacts
TMS roots
audit files
ANN state
database artifacts
```

Relative-path tests execute within isolated temporary workspaces.

The unrelated sprint-note edit remains intentionally preserved.

---

5. Sprint Item #2 — Enforce Collection-Name Safety at the TMS Boundary

The canonical runtime already validates collection names, but direct `HVD15TMSStorage` usage must not bypass this protection.

Wire:

```
validate_collection_name(...)
```

into the actual TMS storage boundary.

At minimum inspect:

```
HVD15TMSStorage._get_or_create_collection(...)
```

plus direct:

```
create
delete
read
write
snapshot
archive
restore
```

paths that construct collection-specific storage paths.

The security boundary should become:

```
API validation
      |
      v
runtime validation
      |
      v
TMS boundary validation
      |
      v
physical storage realization
```

Reject values such as:

```
../../foo
collection/name
.
..
absolute paths
```

```
separator injection
NUL-like invalid forms where applicable
```

before filesystem access.

Acceptance criteria

Tests prove:

```
unsafe name -> rejected before filesystem mutation
safe name   -> behaves exactly as before
direct TMS access cannot bypass validation
```

Suggested test:

```
tests/test_tms_collection_name_safety.py
```

---

6. Sprint Item #3 — Replace the Long HDF5 Barrier With Staged Persistence

The current correctness model effectively behaves like:

```
acquire global consistent-read barrier
        |
        v
walk live TMS state
        |
        v
write entire HDF5 generation
        |
        v
release barrier
```

Correct but potentially disastrous for a large active HVD.

Replace this with:

```
LIVE TMS
   |
```

```
   | short coherent barrier
   v
immutable staging snapshot
   |
   +---- release writers immediately
   |
   v
stable staged state
   |
   +----> HDF5 durable generation
   |
   +----> integrity validation
```

The invariant is:

> **One HDF5 generation represents one coherent logical point in TMS history.**

HDF5 streaming must not require the live system to remain blocked.

Desired abstraction

Conceptually:

```
snapshot = storage.create_consistent_snapshot()

archive.archive_snapshot(
    snapshot,
    destination,
)
```

Do not add a new peer live backend.

Acceptance criteria

Concurrent tests prove:

```
writers pause only during coherent snapshot capture

writers resume while HDF5 continues writing

archive contains one logical point in time

no pre/post snapshot state mixture is possible
```

```
rehydration reconstructs the captured generation exactly
```

Suggested test:

```
tests/test_archive_staging_v15.py
```

---

7. Sprint Item #4 — Scalable Hierarchical Integrity

Per-vector digests become operationally unwieldy at large scale.

Introduce deterministic hierarchical integrity.

Example:

```
archive
 |
 +-- collection A
 |    |
 |    +-- chunk 0000
 |    |     +-- vector digests
 |    |     +-- chunk root
 |    |
 |    +-- chunk 0001
 |    |     +-- vector digests
 |    |     +-- chunk root
 |    |
 |    +-- collection root
 |
 +-- collection B
 |
 +-- archive root
```

Candidate derivation:

```
canonical vector bytes
        |
        v
BLAKE2b/vector digest
        |
        v
```

```
ordered chunk root
        |
        v
collection root
        |
        v
archive root
```

A generalized blockchain-style Merkle subsystem is unnecessary unless justified.

The immediate requirements are:

```
determinism
bounded manifests
partial verification
corruption localization
future snapshot shipping support
future anti-entropy support
```

Acceptance criteria

Tests prove:

```
same state -> identical root

single-vector corruption -> changed chunk/root

corruption can be localized without scanning unrelated chunks

vector ordering/canonicalization is deterministic

archive verification fails closed
```

Suggested test:

```
tests/test_archive_integrity_tree_v15.py
```

---

8. Sprint Item #5 — Explicit Runtime Maintenance and Readiness State

Current `/readyz` behavior does not fully represent whether HVD can safely serve normal traffic during major state transitions.

Introduce an explicit state machine.

Example:

```
STARTING
   |
   v
REBUILDING
   |
   v
READY
   |
   +------> ARCHIVING ------> READY
   |
   +------> REBIRTH --------> REBUILDING
   |
   +------> REINDEXING -----> READY
```

Potential runtime states:

```
STARTING
READY
ARCHIVING
REBIRTH
REINDEXING
DEGRADED
FAILED
```

Potential future recursive-runtime states may later include:

```
FIELD_RECOVERY
ARENA_DRAINING
```

but do not invent them until implementation requires them.

Health semantics

`/healthz`:

> Is the service process alive?

`/readyz`:

> Is this HVD runtime currently safe to receive normal traffic?

During authoritative/destructive restoration:

```
state-sensitive request
        |
        v
503 Service Unavailable
Retry-After: ...
```

Diagnostic endpoints remain available.

Acceptance criteria

Tests prove:

```
REBIRTH never reports READY prematurely

failed archive does not strand runtime indefinitely

health remains reachable during maintenance

mutating requests are correctly gated

runtime transitions are deterministic
```

Suggested test:

```
tests/test_runtime_readiness_v15.py
```

---

9. Sprint Item #6 — Canonicalize the Recursive Cognitive Runtime Contract

This is the new architectural hinge.

Before implementing ephemeral HVD machinery, encode the agreed architecture into the repository.

The canonical project documentation must explicitly state:

```
recursive cognition = dynamic field, not database genealogy

HTCE owns cognitive field/addressability

TMS owns substrate realization/trajectory

HVD owns database semantics

COW != inheritance

TMS trajectory != authority

HIGC = admissibility boundary

THDE = bounded/hysteretic evolution

QMT = continuity boundary

HDF5 = durable long-term/permanent persistence

ANN = derived state
```

The master architecture paper should become the primary architectural reference for this area.

Older documents may remain as historical research, but their status must be clear where they conflict.

Explicitly reject

```
parent_database_id as canonical cognitive identity

child_database_id as canonical cognitive identity

filesystem-path-based HVD identity
```

```
self-promotion/self-persistence semantics

TMS-controlled thought collapse

whole-database placement as a separate future distribution ontology
```

Deliverables

A canonical architecture document under `docs/`.

Optionally a shorter architecture-decision summary that points to it.

Do not delete historical research merely because terminology changed.

Mark supersession clearly.

Acceptance criteria

A new developer should be able to answer correctly:

```
Who creates ephemeral HVDs?            HTCE

Who owns cognitive addressability?     HTCE

Who owns physical state realization?   TMS

Who records trajectory?                TMS

Who defines database semantics?        HVD

Who defines admissible geometry?       HIGC

Who governs bounded/hysteretic motion? THDE

Who handles continuity?                QMT

Does COW imply ancestry?               NO

Are ephemeral HVDs parent/children?    NO
```

---

10. Sprint Item #7 — Define Local and Global Ephemeral Identity

Ephemeral HVDs require both:

```
LOCAL
and
GLOBAL
```

active addressability.

A purely local pointer is insufficient because active cognitive fields may eventually span:

```
threads
processes
NUMA regions
accelerators
nodes
```

A heavy durable database identity on every hot-path lookup may also be unnecessary.

Define three logical identity/address layers.

10.1 LocalEphemeralHandle

Optimized for the hottest path.

Conceptually:

```
LocalEphemeralHandle(
    arena=...,
    generation=...,
    slot=...,
)
```

The exact binary layout is not yet canon.

Requirements:

```
very cheap creation

constant/near-constant local lookup

generation-aware stale-handle rejection

no silent reuse

no filesystem semantics
```

10.2 GlobalEphemeralID

Globally resolves an active ephemeral HVD.

Requirements:

```
globally unique across active runtime

cheap to generate

stable during active lifetime

survives local relocation

may be resolved across process/node boundaries later

cannot silently rebind after reclamation
```

Do not hard-code UUIDv7, ULID, Snowflake, etc. before profiling and design review.

Hide representation behind the abstraction.

10.3 DurableDatabaseID

Used for persistent/materialized HVD identity.

Durable record addressing remains conceptually:

```
(database_id, collection_id, record_id)
```

with:

```
hvd://database_id/collection_id/record_id
```

as the canonical durable logical reference form.

Critical invariant

```
identity != physical path
identity != TMS extent
identity != process
identity != node
identity != shard
identity != CognitiveArena slot
```

Acceptance criteria

Tests prove:

```
local handles resolve correctly

generation mismatch fails closed

global ephemeral IDs are unique

released ID never silently resolves to a later HVD

physical relocation does not change global active identity

durable IDs remain independent of physical realization
```

Suggested tests:

```
tests/cognitive/test_ephemeral_identity.py
tests/cognitive/test_global_ephemeral_id.py
```

---

11. Sprint Item #8 — Implement CognitiveArena

Implement the basic high-rate lifetime primitive for ephemeral HVD populations.

Conceptually:

```
HTCE Thought Epoch / CognitiveArena
 |
 +-- ephemeral HVD 000001
 +-- ephemeral HVD 000002
 +-- ephemeral HVD 000003
 +-- ...
 +-- ephemeral HVD N
```

Creating an ephemeral HVD must **not** automatically create:

```
directory

HDF5 file

standalone WAL

standalone SSTable hierarchy

standalone ANN

process

thread pool

worker service

independent physical Nexus runtime
```

Initial physical representation should be closer to:

```
descriptor
+
local handle
+
global active identity
+
NexusScope reference
+
TMS state view
+
local delta
```

Arena lifecycle

Conceptually:

```
OPEN
 |
 +-- allocate HVDs
 |
 +-- execute cognitive field
 |
 +-- selected state escapes/re-homes if required
 |
 v
CLOSING
 |
 +-- reject unsafe new references
 |
 +-- finalize escaped references
 |
 v
RECLAIMED
```

Do not use the word "promotion" to imply self-declared importance.

Prefer:

```
retain
rehome
materialize
consolidate
```

depending on what actually occurs.

Bulk reclamation

Closing a thought epoch should permit bulk reclamation of non-retained temporary state.

The point is:

```
one arena release
```

rather than:

```
99,997 recursive filesystem deletes
```

Performance requirement

The architectural target remains:

```
100,000+ logical ephemeral HVD births/deaths per second
```

This sprint does **not** claim that performance has already been achieved.

Add benchmark instrumentation now.

Measure:

```
descriptor size
allocations/sec
reclaims/sec
peak metadata
contention
time per bulk teardown
```

Acceptance criteria

Tests prove:

```
spawn does not touch filesystem by default

drop does not individually delete database directories

bulk arena reclamation works

stale local handles fail

retained references survive arena teardown only through explicit escape/rehome

arena teardown cannot silently invalidate escaped state
```

Suggested files:

```
hvd/cognitive/arena.py
tests/cognitive/test_cognitive_arena.py
benchmarks/benchmark_cognitive_arena.py
```

Names are provisional pending repository inspection.

---

12. Sprint Item #9 — HTCE Active Cognitive Field Registry

HTCE needs a first-class representation of the active cognitive field.

This registry manages **cognitive participation**, not physical bytes.

Conceptually:

```
ActiveCognitiveField
```

containing references such as:

```
HVD A
HVD B
HVD C
...
```

through their active handles.

It should support:

```
register potential

remove/release potential

enumerate active population

associate with one or more temporary coalitions

change coalition membership over time

lookup by cognitive handle
```

```
collapse/select candidates

hand retained-state intent to the appropriate lifecycle layer
```

12.1 No permanent graph requirement

Temporary association should not automatically create:

```
A --friend_of--> B
```

as permanent database truth.

A relationship may instead be represented as:

```
R_AB(t)
```

or by hyperdimensional/resonance state.

Coalition membership may conceptually be:

```
c_k(i,t) in [0,1]
```

but this sprint does not need to force one mathematical representation before empirical requirements exist.

12.2 Same HVD, multiple coalitions

The registry must permit:

```
HVD-A:
    coalition alpha
    coalition beta

HVD-B:
    coalition beta

HVD-C:
    no coalition
```

and later:

```
A leaves alpha
B joins gamma
C joins alpha + beta
```

without rewriting database identity.

12.3 HTCE/TMS boundary

HTCE may decide:

```
candidate no longer participates
```

but HTCE should not manipulate:

```
SSTable filenames
WAL offsets
HDF5 chunks
extent refcounts directly
```

It requests lifecycle behavior from the substrate boundary.

Acceptance criteria

Tests prove:

```
field membership changes without identity changes

one HVD can participate in multiple coalitions

coalition membership is not required to be permanent

HTCE can collapse/release field state without direct storage-layout knowledge

TMS does not choose cognitive winners
```

Suggested test:

```
tests/cognitive/test_active_cognitive_field.py
```

---

13. Sprint Item #10 — TMS Structural Sharing and COW Contract

The TMS implementation needs the first explicit mechanism supporting:

```
shared base state
+
candidate-local divergence
```

without implying inheritance.

Conceptually:

```
immutable base B
      |
      +----------+
      |          |
      v          v
    HVD-A      HVD-B
   Delta-A    Delta-B
```

Represent:

```
D_A = B + Delta_A
D_B = B + Delta_B
```

A write to `D_A` must not mutate `B` or `D_B`.

13.1 This is a physical optimization

There is no semantic statement:

```
A is the parent of B
```

The relationship is simply:

```
A and B currently share immutable physical state
```

13.2 TMS realization

This sprint should define a substrate realization contract that can eventually describe state as:

```
resident
cached
device_backed
COW_shared
isolated
remote
replicated
archive_staged
```

Do not necessarily implement every realization during this sprint.

The important part is ensuring that cognitive identity is not coupled to one realization.

13.3 Avoid delta-chain disaster

The implementation should not create unbounded:

```
B + Delta1 + Delta2 + Delta3 + ...
```

chains without a collapse/rebase strategy.

Define at minimum:

```
maximum overlay depth or rebase trigger

extent lifetime ownership

reference-count/epoch interaction

safe materialization path
```

Acceptance criteria

Tests prove:

```
shared immutable state is not copied on spawn

candidate mutation creates local divergence

candidate mutation cannot mutate sibling state

COW relationship requires no parent_database_id

base state survives while still referenced

arena teardown releases unreferenced shared state correctly
```

Suggested test:

```
tests/cognitive/test_tms_structural_sharing.py
```

---

14. Sprint Item #11 — Cross-HVD Reference Resolver

Cross-HVD references are a **hard requirement**.

The resolver must understand at least:

```
local ephemeral handle
global ephemeral ID
durable HVD URI
```

Conceptually:

```
ReferenceResolver
 |
 +-- local resolver
 |
 +-- global-active resolver
```

```
 |
 +-- durable resolver
```

14.1 Durable references

Canonical durable form:

```
hvd://database_id/collection_id/record_id
```

Canonical tuple:

```
(database_id, collection_id, record_id)
```

14.2 Ephemeral references

An active HVD must be able to reference another active HVD without forcing immediate durable materialization.

Example:

```
HVD-A
 |
 +-- active reference --> GlobalEphemeralID(B)
```

14.3 Expiration behavior

If B disappears:

```
resolve(B)
```

must return an explicit result such as:

```
EXPIRED
DATABASE_GONE
UNRESOLVED_EPHEMERAL_REFERENCE
```

Exact naming is implementation-specific.

Never:

```
reuse B's ID
        |
        v
silently resolve to HVD-Z
```

Acceptance criteria

Tests prove:

```
local reference resolves

global active reference resolves

durable URI resolves

relocation does not change durable identity

expired reference fails explicitly

reference never silently rebinds

physical filesystem path never appears in canonical reference identity
```

Suggested test:

```
tests/cognitive/test_cross_hvd_resolver.py
```

---

15. Sprint Item #12 — Lightweight Recursive NexusScope Contract

Every **cognitively complete** HVD must be capable of recursively participating in the Nexus cognitive process.

That means logical access to:

```
CA context
TUCR context
HTCE context
TMS context/view
HIGC context
THDE context
QMT context
```

where appropriate.

It does **not** mean physical duplication.

Rejected:

```
100,000 HVDs
        |
        v
100,000 Python interpreters

100,000 HTCE implementations

100,000 TMS engines

100,000 worker pools
```

Correct direction:

```
shared Nexus implementation
        |
        +-- NexusScope(HVD-A)
        +-- NexusScope(HVD-B)
        +-- NexusScope(HVD-C)
        +-- ...
```

Each scope carries only state that must be logically separate.

15.1 Recursive execution

A cognitively complete HVD may create another cognitive field:

```
HVD-A
 |
 +-- NexusScope-A
       |
       +-- HTCE-A
             |
             +-- CognitiveArena
                    |
                    +-- HVD-X
                    +-- HVD-Y
                    +-- HVD-Z
```

This is **recursive field generation**.

It is not:

```
A parent of X
A parent of Y
A parent of Z
```

Acceptance criteria

Tests prove:

```
NexusScope state can be isolated per HVD

runtime implementation can be shared

one scope can create another field

recursion does not require filesystem nesting

recursive execution does not create semantic parenthood

tearing down an inner arena does not corrupt outer scope state
```

Suggested test:

```
tests/cognitive/test_recursive_nexus_scope.py
```

---

16. Sprint Item #13 — Trajectory Without Identity Freezing

This sprint needs explicit tests around the distinction:

```
current state
trajectory
policy consequence
```

Use a simple synthetic scenario corresponding to the soup-kitchen model.

Example subject:

```
stable-ish dimensions:
    veteran
    Chicagoan
    smoker

dynamic contextual relation:
    Bob relation
    Steve relation
```

Trajectory:

```
Bob:
friend -> enemy -> friend -> enemy -> friend

Steve:
enemy -> friend -> friend -> enemy -> friend
```

The system must be able to retain that historical trajectory without rewriting unrelated state dimensions.

TMS records the sequence.

The sprint does **not** need to implement the complete THDE controller.

Instead provide an interface/hook through which a later THDE implementation can consume historical state.

16.1 No automatic permanent edge

Repeated conflict does not automatically become:

```
enemy_of = TRUE
```

forever.

16.2 Hysteresis-ready history

TMS must preserve enough historical information for a future controller to compute something such as:

```
q_(k+1) = lambda * q_k + severity_k
```

with distinct activation/deactivation thresholds:

```
theta_on > theta_off
```

without TMS itself deciding the intervention.

Acceptance criteria

Tests prove:

```
dynamic relation state changes independently of stable-ish dimensions

trajectory is available after current relation changes

no permanent relation edge is required

TMS records history but does not invoke policy itself

future THDE/HIGC hooks can consume trajectory cleanly
```

Suggested test:

```
tests/cognitive/test_trajectory_without_identity_freezing.py
```

---

17. Explicitly Deferred From This Sprint

The following work is intentionally **not** part of this sprint.

17.1 Full distributed TMS

Do not yet implement:

```
cluster membership protocols
cross-node shard migration
distributed replica installation
cluster-wide anti-entropy
```

The logical identity/addressing model must land first.

17.2 Full consensus

Do not implement full:

```
Raft replicated mutation
quorum authority
leader election as HVD authority
distributed WAL commitment
```

yet.

Consensus before stable identity/placement semantics would be premature.

17.3 Complete HIGC / THDE / QMT

This sprint preserves the architectural boundaries and integration hooks.

It does not attempt to implement the entire Nexus mathematics.

17.4 Full dreaming runtime

Dreaming depends on:

```
CognitiveArena
recursive HVD fields
HTCE expansion
TMS trajectory
HIGC/THDE bounds
QMT continuity
```

This sprint builds the primitives.

It does not claim complete dream execution yet.

17.5 Cognitive truth/authority engine

Do not attempt to turn:

```
similarity
recurrence
replication
persistence
```

into cognitive truth.

That remains explicitly outside database-storage semantics.

---

18. Suggested New Test Layout

Approximate structure:

```
tests/
|
+-- test_tms_collection_name_safety.py
+-- test_archive_staging_v15.py
+-- test_archive_integrity_tree_v15.py
+-- test_runtime_readiness_v15.py
|
+-- cognitive/
```

```
    |
    +-- test_ephemeral_identity.py
    +-- test_global_ephemeral_id.py
    +-- test_cognitive_arena.py
    +-- test_active_cognitive_field.py
    +-- test_tms_structural_sharing.py
    +-- test_cross_hvd_resolver.py
    +-- test_recursive_nexus_scope.py
    +-- test_trajectory_without_identity_freezing.py
```

Final filenames should follow the actual repository conventions discovered during implementation.

---

19. Property-Based and Concurrency Tests

Particularly valuable properties:

```
same ephemeral handle + correct generation
    -> same active object

same slot + wrong generation
    -> never same object

released global ID
    -> never silently rebinds

field membership changes
    -> database identity unchanged

coalition ordering changes
    -> semantics unchanged where order is irrelevant

COW sibling mutation
    -> shared immutable state unchanged

arena teardown
    -> no escaped live reference becomes use-after-free

recursive scope teardown
    -> outer scope remains valid

HDF5 generation
    -> internally coherent point in TMS history
```

```
nearest-neighbor result
    -> cannot mutate authoritative state merely by retrieval
```

Concurrency stress should include:

```
parallel arena allocation

parallel release

cross-thread local handle resolution

simultaneous COW divergence

arena closing while candidates are being retained/re-homed

snapshot staging while active writes continue
```

---

20. Benchmark Instrumentation

Do not wait until final optimization to discover the architecture is too heavy.

Add baseline benchmarks for:

```
ephemeral HVD descriptor allocation

local handle resolution

global ID creation

arena bulk reclamation

COW divergence

shared-base fanout

cross-HVD local resolution

recursive NexusScope creation
```

Record:

```
operations/sec
p50
p95
p99
bytes per logical HVD
peak RSS
allocation count
lock/contention metrics where practical
```

The eventual design target remains:

```
100,000+ ephemeral logical HVD births/deaths per second
```

But this sprint should report the actual baseline rather than falsifying success to meet the target.

---

21. Sprint Exit Criteria

This sprint is complete only when:

```
pytest -q
    -> 0 failures

ruff check hvd tests
    -> 0 findings

git diff --check
    -> clean

git status --short
    -> no test-created artifacts
```

And functionally:

```
[ ] direct TMS access enforces collection-name safety

[ ] HDF5 persistence uses short coherent staging barriers

[ ] HDF5 generation represents a coherent point in TMS history

[ ] archive integrity uses bounded chunk/root verification
```

```
[ ] System Rebirth has explicit readiness/maintenance gating

[ ] recursive HVD architecture is canonicalized in repository docs

[ ] canonical architecture contains no parent/child HVD ontology

[ ] local ephemeral handles exist

[ ] global ephemeral identities exist

[ ] stale ephemeral handles fail closed

[ ] CognitiveArena provides cheap logical HVD creation

[ ] arena teardown supports bulk reclamation

[ ] ephemeral HVD creation does not create filesystem objects by default

[ ] HTCE owns the active cognitive-field registry

[ ] one HVD can participate in multiple temporary coalitions

[ ] coalition changes do not rewrite HVD identity

[ ] TMS owns physical realization rather than cognitive collapse

[ ] COW structural sharing works without inheritance semantics

[ ] sibling HVD mutation cannot modify shared immutable state

[ ] cross-HVD active references resolve

[ ] durable hvd:// references remain physical-location independent

[ ] expired active references never silently rebind

[ ] lightweight NexusScope supports recursive field generation

[ ] recursive execution does not require physical Nexus duplication

[ ] TMS retains trajectory without converting history into permanent identity

[ ] ANN remains explicitly derived/rebuildable

[ ] no full distributed-authority semantics are falsely claimed
```

---

22. The Sprint Immediately After This One

Once the local recursive cognitive-runtime foundation is stable, distribution becomes well-defined.

The next sprint should be:

HVD 1.5 — Distributed Realization of Recursive TMS/HVD State

That sprint can safely implement:

```
infrastructure NodeIdentity

cluster membership

uniform logical placement

durable + globally-active placement keys

HRW / Rendezvous placement

ShardDescriptor

ReplicaSet

deterministic rebalance planning

TMS shard snapshot shipping

integrity-root comparison

anti-entropy primitives

shard-local derived ANN
```

The central invariant becomes:

```
logical HVD identity
        |
        v
physical realization may change
        |
```

```
        +-- local
        +-- remote
        +-- replicated
        +-- rebalanced
        +-- isolated
```

without changing cognitive identity.

Placement should use the same logical model regardless of current HVD size.

For durable state:

```
(database_id, collection_id, record_id)
        |
        v
placement
        |
        v
shard / replica set
```

For globally active ephemeral state, the same distribution architecture can later use the global ephemeral namespace without introducing a second database ontology.

---

23. The Sprint After Distributed Realization

Only after distribution mechanics are deterministic should the project implement:

HVD 1.5 / Nexus 4 — Distributed Authority, Replicated Mutation & Query Execution

That sprint can address:

```
replicated WAL

Raft / quorum mutation ordering

leader/failover semantics

write acknowledgements

read consistency
```

```
partition behavior

replica catch-up

distributed anti-entropy repair

shard-local ANN fanout

global top-K merge/rerank

continuity/admissibility hooks around replicated mutation
```

And even there:

```
replica agreement != truth
```

Raft can establish:

```
these nodes agree on the mutation
```

It cannot independently establish:

```
the mutation is constitutionally admissible
the proposition is true
identity continuity has been preserved
```

Those remain separate Nexus responsibilities.

---

24. Dreaming Follows the Recursive Runtime, Not the Other Way Around

Once:

```
CognitiveArena

HTCE field expansion

recursive NexusScope
```

```
TMS structural sharing

cross-HVD references

trajectory

bounded lifecycle
```

exist, dream execution becomes physically meaningful.

The future dream path becomes approximately:

```
Nexus / Cyrae runtime
        |
        v
reduced-pressure cognitive regime
        |
        v
HTCE field expansion
        |
        +-- ephemeral HVD A
        +-- ephemeral HVD B
        +-- ephemeral HVD C
        +-- ...
                |
                +-- recursively generated fields
        |
        v
HIGC / THDE bounded evolution
        |
        v
HTCE collapse
        |
        +-- release non-selected arenas
        |
        +-- retain relevant resulting state
                |
                v
               TMS
          trajectory / reconsolidation
```

TMS helps preserve the cognitive trajectory.

HTCE performs the generative field expansion and collapse.

The dream candidates do not become waking truth merely because they existed.

---

25. Final Sprint Principle

The old sprint was asking:

> How do we distribute TMS?

The corrected sprint must first answer:

> **What exactly is the recursively cognitive HVD/TMS state that distribution will eventually distribute?**

That requires:

```
identity before placement

field semantics before cluster topology

CognitiveArena before massive spawning

cognitive addressability before distributed routing

structural sharing before recursive scale

trajectory before hysteretic policy

logical NexusScope before recursive execution

local correctness before distributed authority
```

Once those foundations exist, the distribution work from the previous sprint becomes substantially easier because it will finally be moving **well-defined objects with stable semantics** rather than encoding architectural assumptions into shard machinery prematurely.

The development sequence is therefore now:

```
COMPLETED
    HTNLA
    System Rebirth foundation
    engineering cleanup

CURRENT SPRINT
```

```
    persistence hardening
        +
    recursive cognitive-runtime foundation

NEXT SPRINT
    distributed physical realization

FOLLOWING SPRINT
    replicated mutation / distributed authority / query execution

LATER NEXUS INTEGRATION
    full HIGC / THDE / QMT enforcement
    dream execution
    continuity verification
    long-horizon cognitive lifecycle
```

That ordering preserves the architecture we now actually mean to build.